

# **Java Player Registration System**

**Developing Database Applications using JDBC**

Jaturaput Jongsubcharoen

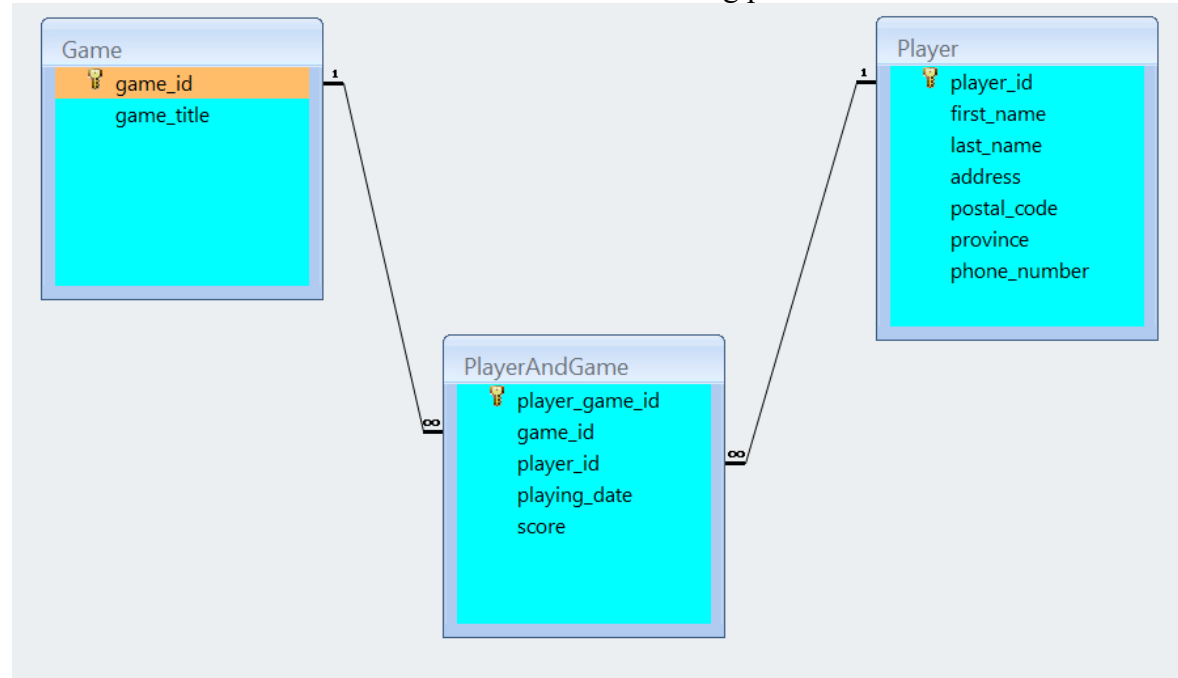
Centennial College

2024, September 11

## **Project Overview**

The Java Player Registration System is a Java-based database application for managing players, games, and gameplay records. The project integrates a JavaFX desktop interface with JDBC and Oracle SQL and was later expanded with a layered architecture, automated testing, CI, and a Spring Boot REST API.

Develop a GUI Java application that will allow the players to submit information about themselves and the games that they are playing on-line. The information will be stored in a simple Oracle database. The database tables are shown in the following picture:



## **Technologies and Tools**

Java 17, JavaFX 21, JDBC, Oracle SQL, Spring Boot 3, Spring Data JPA, Maven, JUnit 5, H2, Docker, GitHub Actions, Git/GitHub

## **Key Features**

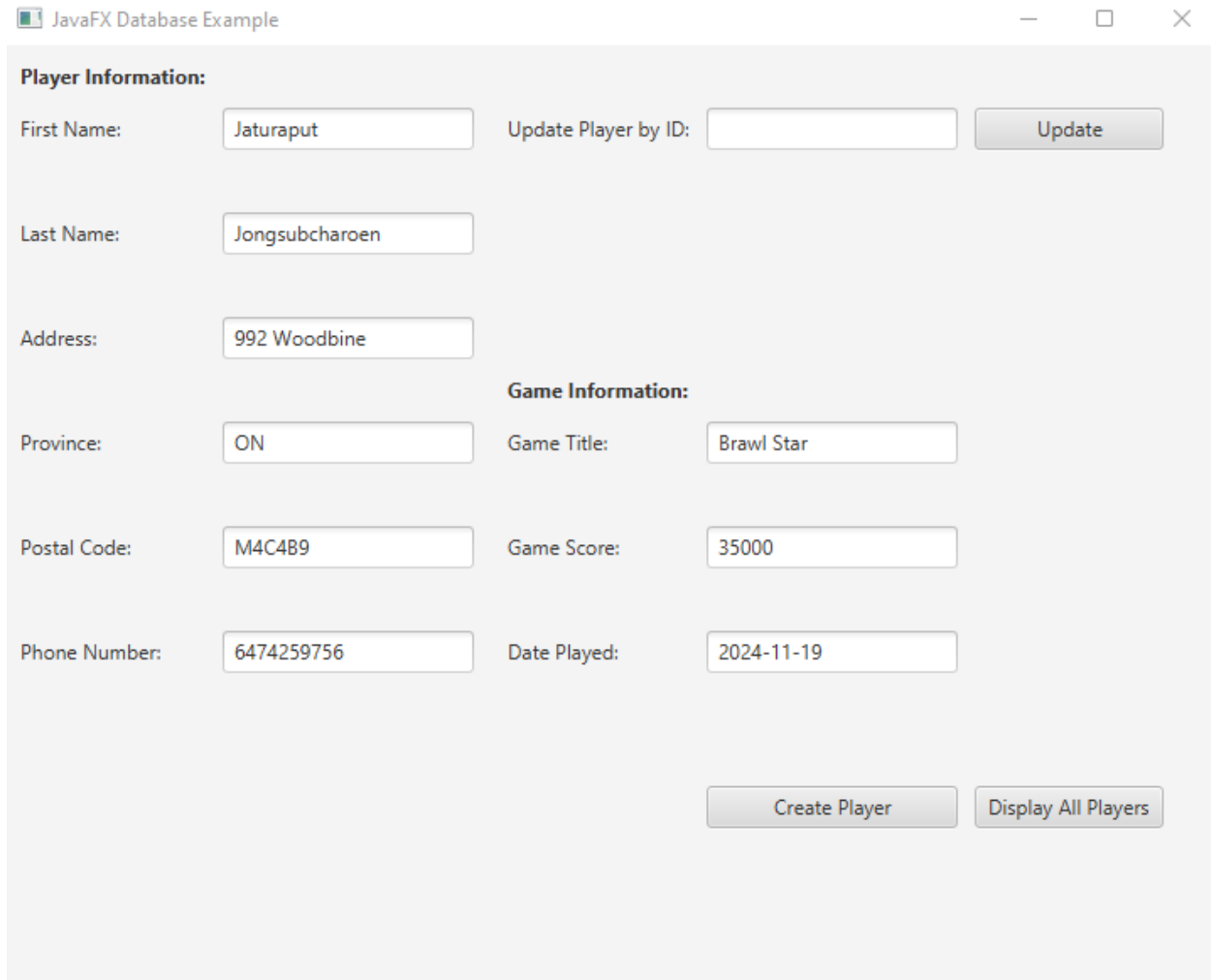
- Player and game creation, updating, deletion, and reporting
- JavaFX desktop user interface with input validation
- JDBC database integration using prepared statements
- Layered Service and DAO architecture
- Transaction management for multi-table operations
- Spring Boot REST API
- JUnit automated testing with H2
- GitHub Actions continuous integration

## **Architecture Diagram**

JavaFX UI → Service Layer → DAO Layer → JDBC → Oracle Database  
Spring Boot REST API → Spring Data JPA → Database

## Solution: Output

**Example: Insert game and player information into the database by clicking Create Player button.**



The screenshot shows a JavaFX application window titled "JavaFX Database Example". The window contains two main sections: "Player Information" and "Game Information".

**Player Information:**

- First Name: Jaturaput
- Last Name: Jongsubcharoen
- Address: 992 Woodbine
- Province: ON
- Postal Code: M4C4B9
- Phone Number: 6474259756

**Game Information:**

- Game Title: Brawl Star
- Game Score: 35000
- Date Played: 2024-11-19

Buttons:

- Update (next to Update Player by ID)
- Create Player
- Display All Players

```

> Start Program ...
> Driver Loaded successfully.
> Player Table created successfully.
> Game Table created successfully.
> PlayerAndGame Table created successfully.
Player Data saved successfully!
Game Data saved successfully!
Player and Game Data saved successfully!

```

[illegible]

```
Player Data saved successfully!  
Game Data saved successfully!  
Player and Game Data saved successfully!  
Player Data saved successfully!  
Game Data saved successfully!  
Player and Game Data saved successfully!  
Data updated successfully!
```

[illegible]

**Example: Update game and player information into the database.**

JavaFX Database Example

Player Information:

First Name:

Elon

Last Name:

Musk

Address:

310 Spadina

Province:

ON

Postal Code:

M5T2E7

Phone Number:

4165919339

Update Player by ID:

1

Update

Game Information:

Game Title:

league of legends

Game Score:

50000

Date Played:

2024-11-20

Create Player

Display All Players

Data updated successfully!

—      □      ×

[illegible]

## Example: Invalid Inputs

JavaFX Database Example

**Player Information:**

First Name:  Update Player by ID:

First Name should contain only letters!

Last Name:

Last Name should contain only letters!

Address:

Address cannot be empty!

**Game Information:**

Province:  Game Title:

Province cannot be empty! Game Title cannot be empty!

Postal Code:  Game Score:

Postal Code cannot be empty! Game Source cannot be empty!

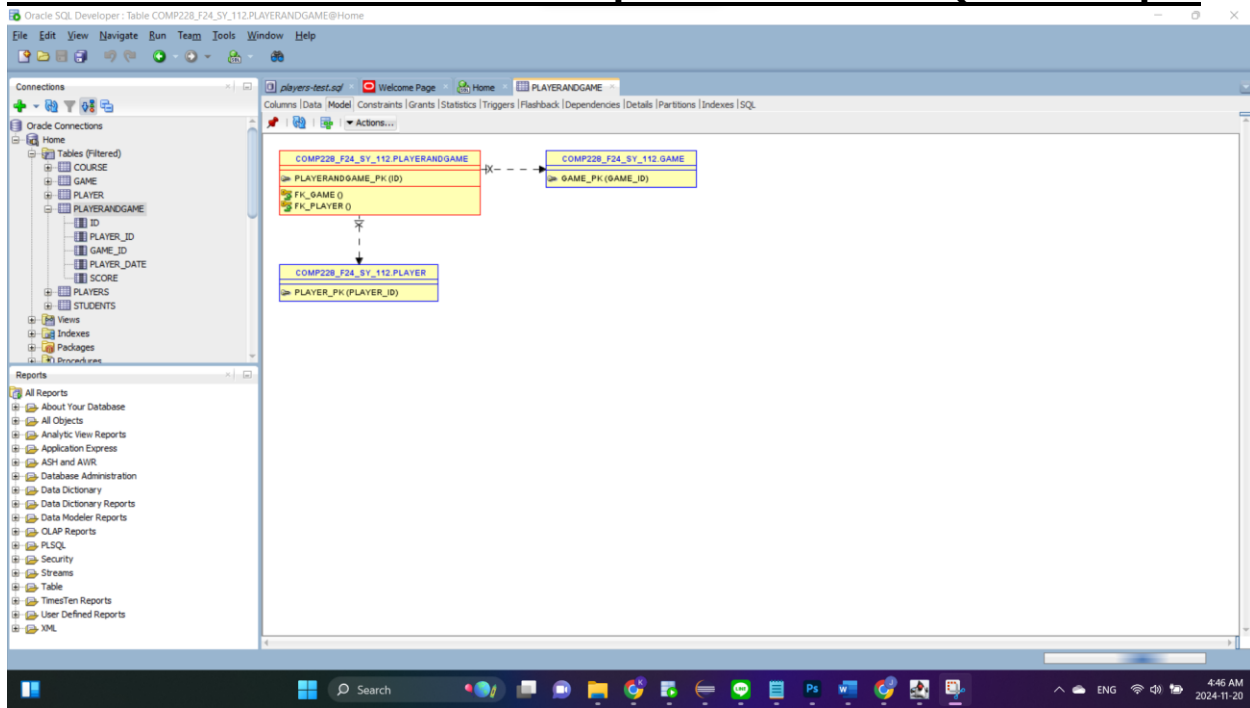
Phone Number:  Date Played:

Phone Number cannot be empty! Date Source cannot be empty!

```
Validation Errors:
Error: First Name should contain only letters!
Error: Last Name should contain only letters!
Error: Address cannot be empty!
Error: Province cannot be empty!
Error: Postal Code cannot be empty!
Error: Phone Number cannot be empty!
Error: Game Title cannot be empty!
Error: Game Source cannot be empty!
Error: Date Source cannot be empty!
```



## Solution of Exercise 1: Model output from Oracle SQL Developer



### Solution: Code

1. *Insert* game and player information into the database.

Insert Game information into the database.

```
private void createGameDatabase() {
    try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASSWORD)) {
        if (conn != null) {
            String createTableSQL = ""
                CREATE TABLE Game (
                    game_id NUMBER GENERATED ALWAYS AS
IDENTITY PRIMARY KEY,
                    game_title VARCHAR2(20) NOT NULL
                )
            """;
            conn.createStatement().execute(createTableSQL);
            System.out.println("> Game Table created successfully.");
        }
    } catch (SQLException e) {
        System.out.println("> Game Table might already exist.");
    }
}

private void saveToGameDatabase(String gameTitle) {
```

```

String insertSQL = "INSERT INTO Game (game_title) VALUES (?)";
try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASSWORD);
    PreparedStatement pstmt = conn.prepareStatement(insertSQL)) {
    pstmt.setString(1, gameTitle);
    pstmt.executeUpdate();
    System.out.println("Game Data saved successfully!");
} catch (SQLException e) {
    e.printStackTrace();
}
}

```

Insert Player in information into the database.

```

private void createPlayerDatabase() {
    try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASSWORD)) {
        if (conn != null) {
            String createTableSQL = ""
                CREATE TABLE Player (
                    player_id NUMBER GENERATED ALWAYS AS IDENTITY
PRIMARY KEY,
                    first_name VARCHAR2(255) NOT NULL,
                    last_name VARCHAR2(255) NOT NULL,
                    address VARCHAR2(255) NOT NULL,
                    province VARCHAR2(2) NOT NULL,
                    postal_code VARCHAR2(6) NOT NULL,
                    phone_number NUMBER(10) NOT NULL
                )
            "";
            conn.createStatement().execute(createTableSQL);
            System.out.println("> Player Table created successfully.");
        }
    } catch (SQLException e) {
        System.out.println("> Player Table might already exist.");
    }
}

private void saveToPlayerDatabase(String firstName, String lastName, String address,
String postalCode, String province, String phoneNumber) {
    String insertSQL = "INSERT INTO Player (first_name, last_name, address, province,
postal_code, phone_number) VALUES (?, ?, ?, ?, ?, ?)";
    try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASSWORD);
        PreparedStatement pstmt = conn.prepareStatement(insertSQL)) {
        pstmt.setString(1, firstName);
        pstmt.setString(2, lastName);
        pstmt.setString(3, address);
        pstmt.setString(4, province);
    }
}

```

```
pstmt.setString(5, postalCode);
pstmt.setString(6, phoneNumber); // Use String for phone number
pstmt.executeUpdate();
System.out.println("Player Data saved successfully!");
} catch (SQLException e) {
    e.printStackTrace();
}
}
```

## 2. Update the existing player information.

```
// Update database
try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASSWORD)) {
    // Update Player table
    String updatePlayerSQL = ""
        UPDATE Player
        SET first_name = ?, last_name = ?, address = ?, province = ?, postal_code = ?,
phone_number = ?
        WHERE player_id = ?
    "";
    try (PreparedStatement pstmt = conn.prepareStatement(updatePlayerSQL)) {
        pstmt.setString(1, firstName);
        pstmt.setString(2, lastName);
        pstmt.setString(3, address);
        pstmt.setString(4, province);
        pstmt.setString(5, postalCode);
        pstmt.setString(6, phoneNumber);
        pstmt.setInt(7, playerId);
        pstmt.executeUpdate();
    }
    // Update Game table
    String updateGameSQL = ""
        UPDATE Game
        SET game_title = ?
        WHERE game_id = (
            SELECT game_id FROM PlayerAndGame WHERE player_id = ?
        )
    "";
    try (PreparedStatement pstmt = conn.prepareStatement(updateGameSQL)) {
        pstmt.setString(1, gameTitle);
        pstmt.setInt(2, playerId);
        pstmt.executeUpdate();
    }
    // Update PlayerAndGame table
    String updatePlayerGameSQL = ""
        UPDATE PlayerAndGame
        SET player_date = ?, score = ?
```

```

        WHERE player_id = ?
        """;
        try (PreparedStatement pstmt =
conn.prepareStatement(updatePlayerGameSQL)) {
            pstmt.setDate(1, datePlayed);
            pstmt.setInt(2, gameScore);
            pstmt.setInt(3, playerId);
            pstmt.executeUpdate();
        }
        System.out.println("Data updated successfully!");
        //System.out.println("Player and Game data updated successfully!");
    } catch (SQLException ex) {
        System.out.println("Error updating data in the database!");
        ex.printStackTrace();
    }
}

```

3. *Display reports with player and played games information. You may use a `JTable` or other components to display the reports. Allow the user to select player\_id.*

```

displayAllPlayersButton.setOnAction(e -> {
    String fetchSQL = ""
        SELECT p.player_id AS ID,
            p.first_name || ' ' || p.last_name AS NAME,
            p.address AS ADDRESS,
            p.postal_code AS POSTAL_CODE,
            p.province AS PROVINCE,
            p.phone_number AS PHONE_NUMBER,
            g.game_title AS GAME_TITLE,
            pg.score AS SCORE,
            pg.player_date AS DATE_PLAYED
        FROM Player p
        JOIN PlayerAndGame pg ON p.player_id = pg.player_id
        JOIN Game g ON pg.game_id = g.game_id
    """;
    try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASSWORD);
        PreparedStatement pstmt = conn.prepareStatement(fetchSQL);
        ResultSet rs = pstmt.executeQuery()) {
        ObservableList<PlayerGameInfo> data = FXCollections.observableArrayList();
        while (rs.next()) {
            data.add(new PlayerGameInfo(
                rs.getInt("ID"),
                rs.getString("NAME"),
                rs.getString("ADDRESS"),
                rs.getString("POSTAL_CODE"),
                rs.getString("PROVINCE"),
                rs.getString("PHONE_NUMBER"),

```

```

        rs.getString("GAME_TITLE"),
        rs.getInt("SCORE"),
        rs.getDate("DATE_PLAYED").toString()
    ));
}
// Create TableView
TableView<PlayerGameInfo> tableView = new TableView<>();
tableView.setItems(data);
// Define Table Columns
TableColumn<PlayerGameInfo, Integer> idCol = new TableColumn<>("ID");
idCol.setCellValueFactory(new PropertyValueFactory<>("id"));
TableColumn<PlayerGameInfo, String> nameCol = new
TableColumn<>("NAME");
nameCol.setCellValueFactory(new PropertyValueFactory<>("name"));
TableColumn<PlayerGameInfo, String> addressCol = new
TableColumn<>("ADDRESS");
addressCol.setCellValueFactory(new PropertyValueFactory<>("address"));
TableColumn<PlayerGameInfo, String> postalCodeCol = new
TableColumn<>("POSTAL CODE");
postalCodeCol.setCellValueFactory(new
PropertyValueFactory<>("postalCode"));
TableColumn<PlayerGameInfo, String> provinceCol = new
TableColumn<>("PROVINCE");
provinceCol.setCellValueFactory(new PropertyValueFactory<>("province"));
TableColumn<PlayerGameInfo, String> phoneNumberCol = new
TableColumn<>("PHONE NUMBER");
phoneNumberCol.setCellValueFactory(new
PropertyValueFactory<>("phoneNumber"));
TableColumn<PlayerGameInfo, String> gameTitleCol = new
TableColumn<>("GAME TITLE");
gameTitleCol.setCellValueFactory(new PropertyValueFactory<>("gameTitle"));
TableColumn<PlayerGameInfo, Integer> scoreCol = new
TableColumn<>("SCORE");
scoreCol.setCellValueFactory(new PropertyValueFactory<>("score"));
TableColumn<PlayerGameInfo, String> datePlayedCol = new
TableColumn<>("DATE PLAYED");
datePlayedCol.setCellValueFactory(new
PropertyValueFactory<>("datePlayed"));
// Add Columns to TableView
tableView.getColumns().addAll(idCol, nameCol, addressCol, postalCodeCol,
provinceCol, phoneNumberCol, gameTitleCol, scoreCol, datePlayedCol);
// Display TableView in a new Scene
Scene tableScene = new Scene(new VBox(tableView), 800, 420);
Stage tableStage = new Stage();
tableStage.setScene(tableScene);
tableStage.setTitle("All Players and Games");
tableStage.show();

```

```
    } catch (SQLException ex) {  
        System.out.println("Error fetching data: " + ex.getMessage());  
        ex.printStackTrace();  
    }  
});
```

### Main class

```
package exercise1;  
//-----  
import javafx.application.Application;  
//-----  
public class Main extends Application {  
    private static final String DB_URL = "jdbc:oracle:thin:@199.212.26.208:1521:SQLD";  
    private static final String DB_USER = "COMP228_F24_sy_112";  
    private static final String DB_PASSWORD = "password";  
    @SuppressWarnings("unchecked")  
    @Override  
    public void start(Stage primaryStage) {  
        try {  
            System.out.println("> Start Program ...");  
            Class.forName("oracle.jdbc.driver.OracleDriver");  
            System.out.println("> Driver Loaded successfully.");  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
        // GridPane for the form  
        GridPane grid = new GridPane();  
        grid.setPadding(new Insets(10));  
        grid.setHgap(10);  
        grid.setVgap(10);  
        // UI Components  
        //-----  
        Label lblPlayerInfo = new Label("Player Information:");  
        lblPlayerInfo.setStyle("-fx-font-weight: bold;");  
  
        Label firstNameLabel = new Label("First Name:");  
        TextField firstNameField = new TextField();  
  
        Label lastNameLabel = new Label("Last Name:");  
        TextField lastNameField = new TextField();  
        Label addressLabel = new Label("Address:");  
        TextField addressField = new TextField();
```

```
Label provinceLabel = new Label("Province:");
TextField provinceField = new TextField();

Label postalCodeLabel = new Label("Postal Code:");
TextField postalCodeField = new TextField();

Label phoneNumberLabel = new Label("Phone Number:");
TextField phoneNumberField = new TextField();

//-----
Label updatePlayerByIdLabel = new Label("Update Player by ID:");
TextField updatePlayerByIdField = new TextField();
Button updateButton = new Button("Update");
updateButton.setMaxWidth(Double.MAX_VALUE);

Label lblGameInfo = new Label("Game Information:");
lblGameInfo.setStyle("-fx-font-weight: bold;");

Label gameTitleLabel = new Label("Game Title:");
TextField gameTitleField = new TextField();

Label gameScoreLabel = new Label("Game Score:");
TextField gameScoreField = new TextField();

Label DatePlayedLabel = new Label("Date Played:");
TextField DatePlayedField = new TextField();

//-----
Button createPlayerButton = new Button("Create Player");
createPlayerButton.setMaxWidth(Double.MAX_VALUE);
Button displayAllPlayersButton = new Button("Display All Players");

//-----
grid.add(lblPlayerInfo, 0, 0);

grid.add(firstNameLabel, 0, 1);
grid.add(firstNameField, 1, 1);
grid.add.lastNameLabel, 0, 3);
grid.add.lastNameField, 1, 3);
grid.add(addressLabel, 0, 5);
grid.add(addressField, 1, 5);
grid.add(provinceLabel, 0, 7);
grid.add(provinceField, 1, 7);
grid.add(postalCodeLabel, 0, 9);
grid.add(postalCodeField, 1, 9);
grid.add(phoneNumberLabel, 0, 11);
```

```
grid.add(phoneNumberField, 1, 11);
/*-----*/
grid.add(updatePlayerByIdLabel, 3, 1);
grid.add(updatePlayerByIdField, 4, 1);
grid.add(updateButton, 5, 1);

grid.add(lblGameInfo, 3, 6);

grid.add(gameTitleLabel, 3, 7);
grid.add(gameTitleField, 4, 7);
grid.add(gameScoreLabel, 3, 9);
grid.add(gameScoreField, 4, 9);
grid.add(DatePlayedLabel, 3, 11);
grid.add(DatePlayedField, 4, 11);

//-----
grid.add(createPlayerButton, 4, 16);
grid.add(displayAllPlayersButton, 5, 16);
//-----

// Create error labels for each field
Label firstNameErrorLabel = new Label();
firstNameErrorLabel.setStyle("-fx-text-fill: red;");
Label lastNameErrorLabel = new Label();
lastNameErrorLabel.setStyle("-fx-text-fill: red;");
Label addressErrorLabel = new Label();
addressErrorLabel.setStyle("-fx-text-fill: red;");
Label postalCodeErrorLabel = new Label();
postalCodeErrorLabel.setStyle("-fx-text-fill: red;");
Label provinceErrorLabel = new Label();
provinceErrorLabel.setStyle("-fx-text-fill: red;");
Label phoneNumberErrorLabel = new Label();
phoneNumberErrorLabel.setStyle("-fx-text-fill: red;");

Label updatePlayerByIdErrorLabel = new Label();
updatePlayerByIdErrorLabel.setStyle("-fx-text-fill: red;");
Label gameTitleErrorLabel = new Label();
gameTitleErrorLabel.setStyle("-fx-text-fill: red;");
Label gameScoreErrorLabel = new Label();
gameScoreErrorLabel.setStyle("-fx-text-fill: red;");
Label dateSourceErrorLabel = new Label();
dateSourceErrorLabel.setStyle("-fx-text-fill: red;");

// Add error labels to the grid
grid.add(firstNameErrorLabel, 0, 2, 2, 1); // Adjacent to First Name field
grid.add(lastNameErrorLabel, 0, 4, 2, 1); // Adjacent to Last Name field
grid.add(addressErrorLabel, 0, 6, 2, 1); // Adjacent to Address field
```



```
grid.add(provinceErrorLabel, 0, 8, 2, 1); // Adjacent to Postal Code field
grid.add(postalCodeErrorLabel, 0, 10, 2, 1); // Adjacent to Province field
grid.add(phoneNumberErrorLabel, 0, 12, 2, 1); // Adjacent to Phone Number field
/*-----*/
grid.add(updatePlayerByIdErrorLabel, 3, 2, 2, 1); // Adjacent to Game Title field

grid.add(gameTitleErrorLabel, 3, 8, 2, 1); // Adjacent to Game Title field
grid.add(gameScoreErrorLabel, 3, 10, 2, 1); // Adjacent to Game Source field
grid.add(dateSourceErrorLabel, 3, 12, 2, 1); // Adjacent to Date Source field

//-----
createPlayerButton.setOnAction(e -> {
    StringBuilder errorMessages = new StringBuilder(); // To collect all error messages
    boolean hasError = false; // Track if there are any errors
    // Clear all previous error messages
    firstNameErrorLabel.setText("");
    lastNameErrorLabel.setText("");
    addressErrorLabel.setText("");
    provinceErrorLabel.setText("");
    postalCodeErrorLabel.setText("");
    phoneNumberErrorLabel.setText("");
    gameTitleErrorLabel.setText("");
    gameScoreErrorLabel.setText("");
    dateSourceErrorLabel.setText("");
    try {
        // Get input values for player
        String firstName = firstNameField.getText();
        String lastName = lastNameField.getText();
        String address = addressField.getText();
        String province = provinceField.getText().trim();
        String postalCode = postalCodeField.getText().trim();
        String phoneNumber = phoneNumberField.getText().trim();
        // Validate first name
        if (firstName.isEmpty()) {
            errorMessages.append("Error: First Name cannot be empty!\n");
            firstNameErrorLabel.setText("First Name cannot be empty!");
            hasError = true;
        } else if (!firstName.matches("[a-zA-Z]+")) {
            errorMessages.append("Error: First Name should contain only letters!\n");
            firstNameErrorLabel.setText("First Name should contain only letters!");
            hasError = true;
        }
        // Validate last name
        if (lastName.isEmpty()) {
            errorMessages.append("Error: Last Name cannot be empty!\n");
            lastNameErrorLabel.setText("Last Name cannot be empty!");
            hasError = true;
        }
    }
});
```

```
    } else if (!lastName.matches("[a-zA-Z]+")) {
        errorMessages.append("Error: Last Name should contain only letters!\n");
        lastNameErrorLabel.setText("Last Name should contain only letters!");
        hasError = true;
    }
    // Validate address
    if (address.isEmpty()) {
        errorMessages.append("Error: Address cannot be empty!\n");
        addressErrorLabel.setText("Address cannot be empty!");
        hasError = true;
    }

    // Validate province
    if (province.isEmpty()) {
        errorMessages.append("Error: Province cannot be empty!\n");
        provinceErrorLabel.setText("Province cannot be empty!");
        hasError = true;
    } else if (!province.matches("[a-zA-Z]{2}")) {
        errorMessages.append("Error: Last Name should contain only letters and be exactly 2
characters long!\n");
        provinceErrorLabel.setText("Last Name should contain only letters and be exactly 2
characters long!");
        hasError = true;
    }
    // Validate postal code
    if (postalCode.isEmpty()) {
        errorMessages.append("Error: Postal Code cannot be empty!\n");
        postalCodeErrorLabel.setText("Postal Code cannot be empty!");
        hasError = true;
    } else if (!postalCode.matches("[a-zA-Z0-9]{6}")) {
        errorMessages.append("Error: Postal Code must be exactly 6 characters and contain
only letters and digits!\n");
        postalCodeErrorLabel.setText("Postal Code must be exactly 6 characters and contain
only letters and digits!");
        hasError = true;
    }
    // Validate phone number
    if (phoneNumber.isEmpty()) {
        errorMessages.append("Error: Phone Number cannot be empty!\n");
        phoneNumberErrorLabel.setText("Phone Number cannot be empty!");
        hasError = true;
    } else if (!phoneNumber.matches("\\d{10}")) { // Regex to ensure exactly 10 digits
        errorMessages.append("Error: Phone Number must be exactly 10 digits and contain
only numbers!\n");
        phoneNumberErrorLabel.setText("Phone Number must be exactly 10 digits and
contain only numbers!");
        hasError = true;
    }
```

```
}

// Get and validate game inputs
String gameTitle = gameTitleField.getText();
if (gameTitle.isEmpty()) {
    errorMessages.append("Error: Game Title cannot be empty!\n");
    gameTitleErrorLabel.setText("Game Title cannot be empty!");
    hasError = true;
}
String gameScoreInput = gameScoreField.getText();
if (gameScoreInput.isEmpty()) {
    errorMessages.append("Error: Game Source cannot be empty!\n");
    gameScoreErrorLabel.setText("Game Source cannot be empty!");
    hasError = true;
} else {
    try {
        int gameSource = Integer.parseInt(gameScoreInput);
        if (gameSource < 0) {
            errorMessages.append("Error: Game Source cannot be negative!\n");
            gameScoreErrorLabel.setText("Game Source cannot be negative!");
            hasError = true;
        }
    } catch (NumberFormatException ex) {
        errorMessages.append("Error: Game Source must contain only digits!\n");
        gameScoreErrorLabel.setText("Game Source must contain only digits!");
        hasError = true;
    }
}
String datePlayedInput = DatePlayedField.getText();
if (datePlayedInput.isEmpty()) {
    errorMessages.append("Error: Date Played cannot be empty!\n");
    dateSourceErrorLabel.setText("Date Played cannot be empty!");
    hasError = true;
} else {
    try {
        Date datePlayed = Date.valueOf(datePlayedInput); // Convert to SQL Date
        Date today = new Date(System.currentTimeMillis()); // Get today's date
        if (datePlayed.after(today)) {
            errorMessages.append("Error: Date Played cannot be in the future!\n");
            dateSourceErrorLabel.setText("Date Played cannot be in the future!");
            hasError = true;
        }
    } catch (IllegalArgumentException ex) {
        errorMessages.append("Error: Date Played must be in YYYY-MM-DD
format!\n");
        dateSourceErrorLabel.setText("Date Played must be in YYYY-MM-DD format!");
        hasError = true;
    }
}
```

```
    }
}
// If there are errors, display messages and stop execution
if (hasError) {
    System.out.println("Validation Errors:\n" + errorMessages.toString());
    return;
} else {
    updatePlayerByIdErrorLabel.setText("");
}

// Save data to database
saveToPlayerDatabase(firstName, lastName, address, postalCode, province,
phoneNumber);
saveToGameDatabase(gameTitle);
int playerId = fetchLastInsertedPlayerId(); // Fetch the most recent player ID
int gameId = fetchLastInsertedGameId(); // Fetch the most recent game ID
saveToPlayerAndGameDatabase(playerId, gameId, Date.valueOf(datePlayedInput),
Integer.parseInt(gameScoreInput));
//System.out.println("Player and game data saved successfully!");
} catch (Exception ex) {
    System.out.println("Error: An unexpected error occurred!");
    ex.printStackTrace();
}
});

//-----
updateButton.setOnAction(e -> {
    StringBuilder errorMessages = new StringBuilder(); // Collect error messages
    boolean hasError = false;

    // Clear previous error messages
    firstNameErrorLabel.setText("");
    lastNameErrorLabel.setText("");
    addressErrorLabel.setText("");
    provinceErrorLabel.setText("");
    postalCodeErrorLabel.setText("");
    phoneNumberErrorLabel.setText("");
    gameTitleErrorLabel.setText("");
    dateSourceErrorLabel.setText("");
    gameScoreErrorLabel.setText("");
    try {
        // Collect inputs
        String idInput = updatePlayerByIdField.getText().trim();
        String firstName = firstNameField.getText().trim();
        String lastName = lastNameField.getText().trim();
        String address = addressField.getText().trim();
        String province = provinceField.getText().trim();
```

```
String postalCode = postalCodeField.getText().trim();
String phoneNumber = phoneNumberField.getText().trim();
String gameTitle = gameTitleField.getText().trim();
String datePlayedInput = DatePlayedField.getText().trim();
String gameScoreInput = gameScoreField.getText().trim();
// Validate inputs
if (idInput.isEmpty()) {
    errorMessages.append("Error: ID cannot be empty!\n");
    updatePlayerByIdErrorLabel.setText("Error: ID cannot be empty!");
    hasError = true; // Set error flag
} else if (!idInput.matches("\\d+")) {
    errorMessages.append("Error: ID must be a valid number!\n");
    updatePlayerByIdErrorLabel.setText("Error: ID must be a valid number!");
    hasError = true; // Set error flag
}
if (firstName.isEmpty() || !firstName.matches("[a-zA-Z]+")) {
    errorMessages.append("Error: First Name should contain only letters!\n");
    firstNameErrorLabel.setText("First Name should contain only letters!");
    hasError = true;
}
if (lastName.isEmpty() || !lastName.matches("[a-zA-Z]+")) {
    errorMessages.append("Error: Last Name should contain only letters!\n");
    lastNameErrorLabel.setText("Last Name should contain only letters!");
    hasError = true;
}
if (address.isEmpty()) {
    errorMessages.append("Error: Address cannot be empty!\n");
    addressErrorLabel.setText("Address cannot be empty!");
    hasError = true;
}
if (province.isEmpty() || !province.matches("[a-zA-Z]{2}")) {
    errorMessages.append("Error: Province must be 2 letters!\n");
    provinceErrorLabel.setText("Province must be 2 letters!");
    hasError = true;
}
if (postalCode.isEmpty() || !postalCode.matches("[a-zA-Z0-9]{6}")) {
    errorMessages.append("Error: Postal Code must be 6 alphanumeric characters!\n");
    postalCodeErrorLabel.setText("Postal Code must be 6 alphanumeric characters!");
    hasError = true;
}
if (phoneNumber.isEmpty() || !phoneNumber.matches("\\d{10}")) {
    errorMessages.append("Error: Phone Number must be 10 digits!\n");
    phoneNumberErrorLabel.setText("Phone Number must be 10 digits!");
    hasError = true;
}
if (gameTitle.isEmpty()) {
    errorMessages.append("Error: Game Title cannot be empty!\n");
```

```
        gameTitleErrorLabel.setText("Game Title cannot be empty!");
        hasError = true;
    }
    if (datePlayedInput.isEmpty()) {
        errorMessages.append("Error: Date Played cannot be empty!\n");
        dateSourceErrorLabel.setText("Date Played cannot be empty!");
        hasError = true;
    } else {
        try {
            Date datePlayed = Date.valueOf(datePlayedInput); // Convert to SQL Date
            Date today = new Date(System.currentTimeMillis()); // Get today's date
            if (datePlayed.after(today)) {
                errorMessages.append("Error: Date Played cannot be in the future!\n");
                dateSourceErrorLabel.setText("Date Played cannot be in the future!");
                hasError = true;
            }
        } catch (IllegalArgumentException ex) {
            errorMessages.append("Error: Date Played must be in YYYY-MM-DD
format!\n");
            dateSourceErrorLabel.setText("Date Played must be in YYYY-MM-DD format!");
            hasError = true;
        }
    }

    if (gameScoreInput.isEmpty()) {
        errorMessages.append("Error: Game Source cannot be empty!\n");
        gameScoreErrorLabel.setText("Game Source cannot be empty!");
        hasError = true;
    } else {
        try {
            int gameSource = Integer.parseInt(gameScoreInput);
            if (gameSource < 0) {
                errorMessages.append("Error: Game Source cannot be negative!\n");
                gameScoreErrorLabel.setText("Game Source cannot be negative!");
                hasError = true;
            }
        } catch (NumberFormatException ex) {
            errorMessages.append("Error: Game Source must contain only digits!\n");
            gameScoreErrorLabel.setText("Game Source must contain only digits!");
            hasError = true;
        }
    }
    if (hasError) {
        System.out.println("Validation Errors:\n" + errorMessages.toString());
        return;
    } else {
        updatePlayerByIdErrorLabel.setText("");
    }
}
```

```
}
int playerId = Integer.parseInt(idInput);
int gameScore = Integer.parseInt(gameScoreInput);
Date datePlayed = Date.valueOf(datePlayedInput);

// Check if player id exists
boolean playerExists = false;
try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASSWORD)) {
    String checkPlayerSQL = "SELECT COUNT(*) AS count FROM Player WHERE
player_id = ?";
    try (PreparedStatement checkStmt = conn.prepareStatement(checkPlayerSQL)) {
        checkStmt.setInt(1, playerId);
        try (ResultSet rs = checkStmt.executeQuery()) {
            if (rs.next() && rs.getInt("count") > 0) {
                playerExists = true;
            }
        }
    }
} catch (SQLException ex) {
    System.out.println("Error checking player existence in the database!");
    ex.printStackTrace();
    return;
}

if (!playerExists) {
    updatePlayerByIdErrorLabel.setText("Error: Player ID does not exist!");
    System.out.println("Error: Player ID does not exist!");
    return;
} else {
    updatePlayerByIdErrorLabel.setText("Data updated successfully!");
    System.out.println("Data updated successfully!");
}

// Update database
try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASSWORD)) {
    // Update Player table
    String updatePlayerSQL = ""
        UPDATE Player
        SET first_name = ?, last_name = ?, address = ?, province = ?, postal_code = ?,
phone_number = ?
        WHERE player_id = ?
    "";
    try (PreparedStatement pstmt = conn.prepareStatement(updatePlayerSQL)) {
        pstmt.setString(1, firstName);
        pstmt.setString(2, lastName);
        pstmt.setString(3, address);
        pstmt.setString(4, province);
```



```

        pstmt.setString(5, postalCode);
        pstmt.setString(6, phoneNumber);
        pstmt.setInt(7, playerId);
        pstmt.executeUpdate();
    }
    // Update Game table
    String updateGameSQL = ""
        UPDATE Game
        SET game_title = ?
        WHERE game_id = (
            SELECT game_id FROM PlayerAndGame WHERE player_id = ?
        )
    """;
    try (PreparedStatement pstmt = conn.prepareStatement(updateGameSQL)) {
        pstmt.setString(1, gameTitle);
        pstmt.setInt(2, playerId);
        pstmt.executeUpdate();
    }
    // Update PlayerAndGame table
    String updatePlayerGameSQL = ""
        UPDATE PlayerAndGame
        SET player_date = ?, score = ?
        WHERE player_id = ?
    """;
    try (PreparedStatement pstmt = conn.prepareStatement(updatePlayerGameSQL)) {
        pstmt.setDate(1, datePlayed);
        pstmt.setInt(2, gameScore);
        pstmt.setInt(3, playerId);
        pstmt.executeUpdate();
    }

    } catch (SQLException ex) {
        System.out.println("Error updating data in the database!");
        ex.printStackTrace();
    }
    } catch (Exception ex) {
        System.out.println("Error processing update!");
        ex.printStackTrace();
    }
    });
    //-----
    -----
    displayAllPlayersButton.setOnAction(e -> {
        String fetchSQL = ""
            SELECT p.player_id AS ID,
                p.first_name || ' ' || p.last_name AS NAME,
                p.address AS ADDRESS,

```



```

        p.postal_code AS POSTAL_CODE,
        p.province AS PROVINCE,
        p.phone_number AS PHONE_NUMBER,
        g.game_title AS GAME_TITLE,
        pg.score AS SCORE,
        pg.player_date AS DATE_PLAYED
    FROM Player p
    JOIN PlayerAndGame pg ON p.player_id = pg.player_id
    JOIN Game g ON pg.game_id = g.game_id
    WHERE pg.player_date = (
        SELECT MAX(player_date)
        FROM PlayerAndGame
        WHERE player_id = p.player_id
    )
    """;
    try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASSWORD);
        PreparedStatement pstmt = conn.prepareStatement(fetchSQL);
        ResultSet rs = pstmt.executeQuery()) {

        System.out.println("Database connection successful. Fetching data...");

        ObservableList<PlayerGameInfo> data = FXCollections.observableArrayList();

        while (rs.next()) {
            data.add(new PlayerGameInfo(
                rs.getInt("ID"),
                rs.getString("NAME"),
                rs.getString("ADDRESS"),
                rs.getString("POSTAL_CODE"),
                rs.getString("PROVINCE"),
                rs.getString("PHONE_NUMBER"),
                rs.getString("GAME_TITLE"),
                rs.getInt("SCORE"),
                rs.getDate("DATE_PLAYED").toString()
            ));
        }
        // Create TableView
        TableView<PlayerGameInfo> tableView = new TableView<>();
        tableView.setItems(data);
        // Define Table Columns
        TableColumn<PlayerGameInfo, Integer> idCol = new TableColumn<>("ID");
        idCol.setCellValueFactory(new PropertyValueFactory<>("id"));
        TableColumn<PlayerGameInfo, String> nameCol = new TableColumn<>("NAME");
        nameCol.setCellValueFactory(new PropertyValueFactory<>("name"));
        TableColumn<PlayerGameInfo, String> addressCol = new
TableColumn<>("ADDRESS");

```

```

        addressCol.setCellValueFactory(new PropertyValueFactory<("address")>);
        TableColumn<PlayerGameInfo, String> postalCodeCol = new
TableColumn<("POSTAL CODE");
        postalCodeCol.setCellValueFactory(new PropertyValueFactory<("postalCode")>);
        TableColumn<PlayerGameInfo, String> provinceCol = new
TableColumn<("PROVINCE");
        provinceCol.setCellValueFactory(new PropertyValueFactory<("province")>);
        TableColumn<PlayerGameInfo, String> phoneNumberCol = new
TableColumn<("PHONE NUMBER");
        phoneNumberCol.setCellValueFactory(new
PropertyValueFactory<("phoneNumber")>);
        TableColumn<PlayerGameInfo, String> gameTitleCol = new TableColumn<("GAME
TITLE");
        gameTitleCol.setCellValueFactory(new PropertyValueFactory<("gameTitle")>);
        TableColumn<PlayerGameInfo, Integer> scoreCol = new TableColumn<("SCORE");
        scoreCol.setCellValueFactory(new PropertyValueFactory<("score")>);
        TableColumn<PlayerGameInfo, String> datePlayedCol = new TableColumn<("DATE
PLAYED");
        datePlayedCol.setCellValueFactory(new PropertyValueFactory<("datePlayed")>);
        // Add Columns to TableView
        tableView.getColumns().addAll(idCol, nameCol, addressCol, postalCodeCol,
provinceCol, phoneNumberCol, gameTitleCol, scoreCol, datePlayedCol);
        /*-----*/
        // Add a row click listener
        tableView.setRowFactory(tv -> {
            TableRow<PlayerGameInfo> row = new TableRow<>();
            row.setOnMouseClicked(event -> {
                if (event.getClickCount() == 2 && !row.isEmpty()) {
                    PlayerGameInfo selectedPlayer = row.getItem();
                    // Confirm deletion
                    Alert alert = new Alert(Alert.AlertType.CONFIRMATION);
                    alert.setTitle("Delete Confirmation");
                    alert.setHeaderText(null);
                    alert.setContentText("Are you sure you want to delete Player ID: " +
selectedPlayer.getId() + "?");
                    alert.showAndWait().ifPresent(response -> {
                        if (response == ButtonType.OK) {
                            try (Connection deleteConn = DriverManager.getConnection(DB_URL,
DB_USER, DB_PASSWORD)) {
                                // Delete from Player table
                                String deletePlayerSQL = "DELETE FROM Player WHERE player_id =
?";
                                try (PreparedStatement deleteStmt =
deleteConn.prepareStatement(deletePlayerSQL)) {
                                    deleteStmt.setInt(1, selectedPlayer.getId());
                                    deleteStmt.executeUpdate();

```

```

        System.out.println("Player ID " + selectedPlayer.getId() + " deleted
successfully.");
    }
    // Clean up orphaned games
    String deleteUnusedGamesSQL = ""
        DELETE FROM Game
        WHERE game_id NOT IN (SELECT game_id FROM
PlayerAndGame)
    "";
    try (PreparedStatement deleteGamesStmt =
deleteConn.prepareStatement(deleteUnusedGamesSQL)) {
        int rowsAffected = deleteGamesStmt.executeUpdate();
        System.out.println(rowsAffected + " unused games deleted.");
    }
    // Remove from ObservableList
    data.remove(selectedPlayer);
} catch (SQLException ex) {
    System.out.println("Error deleting player or cleaning up games: " +
ex.getMessage());
    ex.printStackTrace();
}
}
});
});
return row;
});

/*-----*/
// Display TableView in a new Scene
Scene tableScene = new Scene(new VBox(tableView), 900, 420);
Stage tableStage = new Stage();
tableStage.setScene(tableScene);
tableStage.setTitle("All Players and Games");
tableStage.show();
} catch (SQLException ex) {
    System.out.println("Error fetching data: " + ex.getMessage());
    ex.printStackTrace();
}
});
/*-----*/
// Scene and Stage
Scene scene = new Scene(grid, 720, 550);
primaryStage.setTitle("JavaFX Database Example");
primaryStage.setScene(scene);
primaryStage.show();
// Initialize the database

```

```

createPlayerDatabase();
createGameDatabase();
createPlayerAndGameDatabase();
}

/*-----*/
private void createPlayerDatabase() {
    try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASSWORD)) {
        if (conn != null) {
            String createTableSQL = ""
CREATE TABLE Player (
    player_id NUMBER GENERATED ALWAYS AS IDENTITY
PRIMARY KEY,
    first_name VARCHAR2(255) NOT NULL,
    last_name VARCHAR2(255) NOT NULL,
    address VARCHAR2(255) NOT NULL,
    province VARCHAR2(2) NOT NULL,
    postal_code VARCHAR2(6) NOT NULL,
    phone_number NUMBER(10) NOT NULL
)
            "";
            conn.createStatement().execute(createTableSQL);
            System.out.println("> Player Table created successfully.");
        }
    } catch (SQLException e) {
        System.out.println("> Player Table might already exist.");
    }
}

private void saveToPlayerDatabase(String firstName, String lastName, String address, String
postalCode, String province, String phoneNumber) {
    String insertSQL = "INSERT INTO Player (first_name, last_name, address, province,
postal_code, phone_number) VALUES (?, ?, ?, ?, ?, ?)";
    try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASSWORD);
        PreparedStatement pstmt = conn.prepareStatement(insertSQL)) {
        pstmt.setString(1, firstName);
        pstmt.setString(2, lastName);
        pstmt.setString(3, address);
        pstmt.setString(4, province);
        pstmt.setString(5, postalCode);
        pstmt.setString(6, phoneNumber); // Use String for phone number
        pstmt.executeUpdate();
        System.out.println("Player Data saved successfully!");
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

```

    }
    /*-----*/
    private void createGameDatabase() {
        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASSWORD)) {
            if (conn != null) {
                String createTableSQL = ""
                    CREATE TABLE Game (
                        game_id NUMBER GENERATED ALWAYS AS
IDENTITY PRIMARY KEY,
                        game_title VARCHAR2(20) NOT NULL
                    )
                """;
                conn.createStatement().execute(createTableSQL);
                System.out.println("> Game Table created successfully.");
            }
        } catch (SQLException e) {
            System.out.println("> Game Table might already exist.");
        }
    }

    private void saveToGameDatabase(String gameTitle) {
        String insertSQL = "INSERT INTO Game (game_title) VALUES (?)";
        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASSWORD);
            PreparedStatement pstmt = conn.prepareStatement(insertSQL)) {
            pstmt.setString(1, gameTitle);
            pstmt.executeUpdate();
            System.out.println("Game Data saved successfully!");
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
    /*-----*/

    private void createPlayerAndGameDatabase() {
        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASSWORD)) {
            if (conn != null) {
                String createTableSQL = ""
                    CREATE TABLE PlayerAndGame (
                        id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
                        player_id NUMBER NOT NULL,
                        game_id NUMBER NOT NULL,
                        player_date DATE NOT NULL,
                        score NUMBER NOT NULL,
                        CONSTRAINT fk_player FOREIGN KEY (player_id) REFERENCES
Player(player_id) ON DELETE CASCADE,
                """;
                conn.createStatement().execute(createTableSQL);
                System.out.println("> Player and Game Table created successfully.");
            }
        } catch (SQLException e) {
            System.out.println("> Player and Game Table might already exist.");
        }
    }
}

```

```

        CONSTRAINT fk_game FOREIGN KEY (game_id) REFERENCES
Game(game_id) ON DELETE CASCADE
    )
    """;
    conn.createStatement().execute(createTableSQL);
    System.out.println("> PlayerAndGame Table created successfully.");
}
} catch (SQLException e) {
    System.out.println("> PlayerAndGame Table might already exist.");
    //e.printStackTrace(); // Print the stack trace for debugging
}
}

private void saveToPlayerAndGameDatabase(int playerId, int gameId, Date playerDate, int
score) {
    String insertSQL = "INSERT INTO PlayerAndGame (player_id, game_id, player_date, score)
VALUES (?, ?, ?, ?)";
    try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASSWORD);
        PreparedStatement pstmt = conn.prepareStatement(insertSQL)) {
        pstmt.setInt(1, playerId);
        pstmt.setInt(2, gameId);
        pstmt.setDate(3, playerDate);
        pstmt.setInt(4, score);
        pstmt.executeUpdate();
        System.out.println("Player and Game Data saved successfully!");
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

/*-----*/
private int fetchLastInsertedPlayerId() {
    String query = "SELECT MAX(player_id) AS last_id FROM Player";
    try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASSWORD);
        PreparedStatement pstmt = conn.prepareStatement(query);
        ResultSet rs = pstmt.executeQuery()) {
        if (rs.next()) {
            return rs.getInt("last_id");
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return -1; // Return -1 if no ID is found
}

private int fetchLastInsertedGameId() {
    String query = "SELECT MAX(game_id) AS last_id FROM Game";

```

```
try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASSWORD);
    PreparedStatement pstmt = conn.prepareStatement(query);
    ResultSet rs = pstmt.executeQuery()) {
    if (rs.next()) {
        return rs.getInt("last_id");
    }
} catch (SQLException e) {
    e.printStackTrace();
}
return -1; // Return -1 if no ID is found
}
```

### PlayerGameInfo class

```
package exercise1;
public class PlayerGameInfo {
    private int id;
    private String name;
    private String address;
    private String postalCode;
    private String province;
    private String phoneNumber;
    private String gameTitle;
    private int score;
    private String datePlayed;
    public PlayerGameInfo(int id, String name, String address, String postalCode, String province,
String phoneNumber, String gameTitle, int score, String datePlayed) {
        this.id = id;
        this.name = name;
        this.address = address;
        this.postalCode = postalCode;
        this.province = province;
        this.phoneNumber = phoneNumber;
        this.gameTitle = gameTitle;
        this.score = score;
        this.datePlayed = datePlayed;
    }
    // Getters for TableView
    public int getId() { return id; }
    public String getName() { return name; }
    public String getAddress() { return address; }
    public String getPostalCode() { return postalCode; }
    public String getProvince() { return province; }
    public String getPhoneNumber() { return phoneNumber; }
```

```
public String getGameTitle() { return gameTitle; }  
public int getScore() { return score; }  
public String getDatePlayed() { return datePlayed; }  
}
```